SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		

Student Name: SUNDARESAN.R.U. **Reg. No.:** 192424359

Section / Batch: 2024 **Date:** 20/07/2026

Code of Conduct

I SUNDARESAN.R.U. (192424359) *(Name / Reg No)*

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sundaler R.U.

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:		**100 Marks**

construct an efficient system using two stacks with a single array structure to maximize memory utilization. compare fixed partitioning and dynamic allocation strategies and analyze how each approach affect overflow condition space efficiency and operational complexity under varying workload.

1) Two stacks in a single array
Concept: Two stacks share one array, one stack grow from the left and the other grows from the right maximizing memory utilization.

→ Fixed partitioning:
* Array is divided into two fixed halves
* each stack has a predefined size

→ Advantages:
* Simple implementation
* Push and pop operations take O(1) time

→ Disadvantages:
* one stack may overflow even if the other has free space
* poor space utillization.

→ Dynamic allocation
* Both stacks share the entire array
* Stack 1 grow from the left, stack 2 grows meets right
* overflow occurs only when both stacks meets.

→ Advantages:
* Better memory utilization
* Delays overflow.
* efficient under uneven workload

→ Disadvantages:
* Slightly more complex implementation.

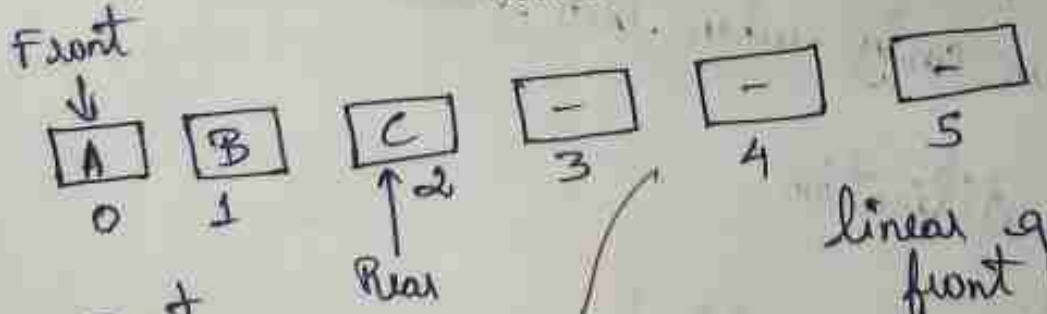
push and pop operation are still O(1).

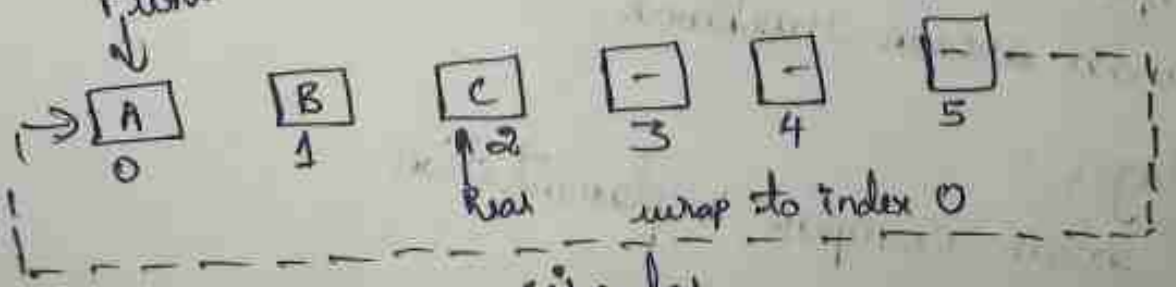
feature	fixed partition	Dynamic allocation
memory utilization	low	high
overflow	early	only when any full
Space efficiency	poor	excellent
push/pop complexity	O(1)	O(1)
Best for	Balanced workloads	Variable workload

⇒ conclusion:
Dynamic allocation is more efficient because maximizes Space usage and reduces unnecessary overflow.

⇒ A queue based ticketing system queues customer in the order of arrival but VIP customer must be prioritized occassionaly. analyze how modifying a simple queue into a priority based queue affects fairness waiting time and computational complexity compare both system using analytical reasoning

Simple Queue Vs priority Queue





A simple Queue follows FIFO (First in, First out) serving customers in arrival order.

A priority Queue serve VIP customer first based on priority regardless of arrival time

feature	Simple Queue	priority Queue
order	FIFO	priority based
fairness	High	Low (VIPs jump ahead)
waiting time	equal for all	reduced for VIPs, increased for other.
Insert complexity	O(1)	O(log n) with heap
Delete complexity	O(n)	O(log n) with heap)

→ Analysis :

* Simple Queue : fair and easy to implement but cannot prioritize urgent customer.
* priority Queue : Improve service for VIPS but reduces fairness and increase computation complexity.

conclusion :

use a simple queue when fairness is the priority use a priority Queue when urgent or VIP customer must receive faster service.